

Development of a Modular Autonomous PuzzleBot Architecture Integrating ROS 2, Embedded Control, Perception, and Intelligent Interfaces

Juan Manuel Félix Campos, José Luis Urquieta Aguilar, Samuel Francisco Cabrera Ramírez,
Alejandro Rodríguez del Bosque, and Eduardo Hernández Valdez
Tecnológico de Monterrey, Monterrey, Mexico
Email: {A0083805, A00835580, A00838072, A01722329, A01285233}@tec.mx

Abstract—This work presents the development and integration of a modular autonomous PuzzleBot architecture designed for mobile robotics tasks involving navigation, perception, actuation, embedded control, and intelligent human-robot interaction. The system was implemented using ROS 2 as the main middleware, enabling distributed communication between perception, control, navigation, hardware, and behavior modules. Visual perception capabilities were incorporated through camera based processing, including QR and fiducial-marker detection, to support environment interpretation and task execution. Autonomous behavior was addressed through localization, odometry, trajectory control, and mission level coordination, allowing the robot to operate beyond manual teleoperation.

A key contribution of the project was the integration of an embedded forklift control subsystem. This mechanism combined a geared DC motor with encoder feedback, FPGA-based PWM generation, quadrature encoder counting, and SPI communication with the high-level computing layer. This allowed the PuzzleBot to connect physical actuation with the autonomous software stack. Additionally, the system incorporated intelligent interaction interfaces, including a web-based HMI for monitoring and control, and an offline voice interface for command execution through a ROS 2 bridge.

The final result was an integrated robotic platform that combined emerging technologies, embedded systems, autonomous navigation, and intelligent interfaces. Integration tests validated the interaction between software modules, perception tools, actuator control, and user interfaces, demonstrating the feasibility of a complete modular robotics architecture.

Index Terms—PuzzleBot, ROS 2, autonomous mobile robot, embedded control, FPGA, SPI communication, computer vision, human-robot interaction, web-based HMI, voice interface.

I. INTRODUCTION

Mobile robotics challenges require more than the independent development of algorithms or mechanical subsystems. A functional autonomous robot must integrate sensing, perception, control, embedded electronics, communication protocols, user interfaces, and physical actuation into a coordinated manipulation. The integrated PuzzleBot platform, shown in Fig. 1, was used as a differential drive mobile robot for the development of an integrated

robotic system capable of autonomous navigation, visual perception, actuator control, intelligent interaction, and forklift-based manipulation.

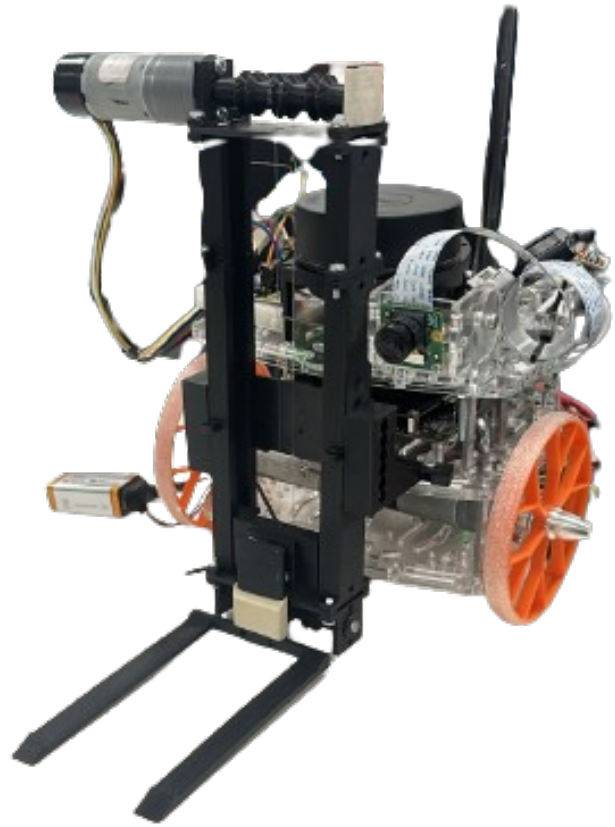


Fig. 1. The integrated PuzzleBot platform equipped with the custom forklift mechanism, Jetson Nano, and sensor suite.

The main objective of this project was to design and integrate a modular robotic architecture that allowed PuzzleBot to operate as a complete autonomous system rather than as a set of isolated components. The robot was developed around ROS 2 as the main middleware, enabling communication between perception, navigation, control, hardware, and behavior modules. Through this architecture, sensor information could be processed by perception algorithms, transformed into localization and navigation decisions, and finally executed through low level control commands.

The technical challenge of the project was to transform individual components sensors, actuators, camera based perception, ROS 2 nodes, FPGA logic, communication interfaces, and user interaction tools into a functional and coordinated robotic platform. This required integrating multiple layers of the system, including visual detection through QR and fiducial markers, odometry and localization, trajectory control, mission level behavior, web based monitoring, voice command interaction, and embedded forklift actuation.

The scope of the project was aligned with four major learning modules. Emerging technologies were addressed through the use of ROS 2, computer vision, Docker based development, and robotic visualization tools. Embedded systems were represented by the FPGA based forklift controller, PWM motor control, quadrature encoder feedback, SPI communication, and actuator integration. Autonomous robots were addressed through localization, navigation, path-following, perception based alignment, and finite state mission logic. Finally, intelligent interaction through hardware and software interfaces was implemented through a web based human machine interface, teleoperation tools, visualization dashboards, and an offline voice user interface connected to the ROS 2 ecosystem.

Overall, this project focused on the integration of a complete robotic system where software, hardware, embedded control, perception, autonomy, and human robot interaction were designed to work together. This approach reflects the central purpose of robotics and digital systems integration: building a system in which each subsystem contributes to a larger autonomous behavior.

II. SYSTEM OVERVIEW AND REPOSITORY ORGANIZATION

The PuzzleBot project was organized as a distributed robotic development ecosystem, where each repository represented a specific technical layer of the complete system. Instead of treating the repositories as isolated codebases, the architecture was structured around the integration of high level autonomy, embedded actuation, perception, communication, user interaction, and hardware design. This organization allowed the team to develop and validate subsystems independently while maintaining a common objective: building a coordinated autonomous mobile robot.

The main integration layer was implemented in the PuzzleBot software repository, which contained the ROS 2 workspace used to connect perception, navigation, control, behavior, hardware interfaces, and custom messages. This repository acted as the central software backbone of the system because it defined the main robotic pipeline from sensor data acquisition to decision making and actuator command generation. The autonomy stack was complemented by localization, mapping, and optimization experiments developed in the PuzzleBot localization repository.

The embedded control layer was developed through the PuzzleBot embedded repository. This subsystem focused on FPGA based actuator control, including PWM generation, quadrature encoder counting, motor direction logic, SPI communication, and low level interaction with the forklift mechanism. In

the final architecture, the Jetson computer performs high level computation and sends commands to the FPGA through SPI, while the FPGA executes real time motor control and processes encoder feedback. This separation improves the structure of the system by assigning computationally intensive robotic decisions to the main processor and deterministic actuator control to the embedded layer.

The interaction and supervision layer was distributed across the voice user interface, web interface for PuzzleBot, and PuzzleBot communications repositories. The voice interface provided an offline human robot interaction mechanism capable of recognizing isolated commands and forwarding them into the ROS 2 ecosystem. The web interface provided a human machine interface for monitoring and control through a frontend, backend, ROS bridge, and robot related visualization tools. The communications repository supported teleoperation, client server interaction, and auxiliary testing tools, which were important during integration and debugging.

Finally, the ROSNin Electronics repository documented the electronic design layer of the system, including PCB related work and hardware integration resources. This layer supported the physical implementation of the robot by connecting the software and embedded logic with real electrical components.

The repository organization also supported a technology readiness level perspective. Software components such as ROS 2 nodes, perception algorithms, web tools, and voice commands progressed from isolated functional prototypes toward integrated system level validation. Hardware components such as the FPGA controller, motor driver, encoder feedback, SPI link, and forklift mechanism progressed from component level testing toward a more complete embedded actuation subsystem. Therefore, the overall architecture reflects not only code organization, but also the maturity progression of the robot from individual subsystem validation to integrated robotic operation.

Table I summarizes the role of each repository within the complete PuzzleBot architecture.

III. EMERGING TECHNOLOGIES AND TECHNOLOGY READINESS

A. ROS 2 as the Integration Middleware

The PuzzleBot architecture was developed around ROS 2 Humble as the main integration middleware for the robotic system. ROS 2 provided the communication infrastructure required to connect perception, localization, navigation, control, hardware interfaces, embedded actuation, and user interaction tools within a distributed architecture. Instead of implementing each functionality as an isolated program, the system

was structured through independent nodes that exchanged information using topics, messages, and coordinated launch files.

This middleware based approach allowed the project to maintain modularity and separation of responsibilities between subsystems. Perception nodes were responsible for processing camera-based information, control nodes handled motion and actuator commands, navigation

TABLE I
REPOSITORY CONTRIBUTION WITHIN THE PUZZLEBOT ARCHITECTURE

Repository	Main contribution	Technical layer
PuzzleBot software	ROS 2 architecture, perception, navigation, control, behavior logic, hardware interfaces, and custom messages	Main autonomy and integration layer
PuzzleBot embedded	FPGA control, PWM generation, encoder feedback, SPI communication, IMU integration, and forklift actuator control	Embedded control layer
Voice user interface	Offline voice recognition, command interpretation, and ROS 2 bridge	Human robot interaction layer
Web interface for PuzzleBot	Web dashboard, backend, frontend, ROS bridge, and robot monitoring tools	Human machine interface layer
PuzzleBot localization	Mapping, localization, and optimization experiments	Localization and navigation support layer
PuzzleBot communications	Teleoperation, communication scripts, client server tools, and auxiliary testing utilities	Communication and testing layer
ROSNin Electronics	PCB design and electronic integration resources	Electrical hardware layer

components supported autonomous movement, and hardware interface nodes connected the software stack with physical devices. This organization made the system easier to debug, extend, and integrate because each component could be tested individually before being connected to the complete robotic pipeline.

The use of ROS 2 also supported scalability. As additional subsystems were developed, such as the forklift controller, web interface, voice interface, QR detection, fiducial marker detection, and visualization tools, they could be integrated into the same communication ecosystem without redesigning the complete architecture. This was especially important because the project required the coordination of heterogeneous technologies, including high level robotic decision making, embedded FPGA control, sensor feedback, and human robot interaction.

Docker was incorporated as part of the development environment to improve reproducibility across different machines. By containerizing the ROS 2 Humble workspace, the team reduced dependency conflicts and made it easier to execute the same software stack in a consistent environment. This was relevant during integration because robotic systems usually depend on specific versions of libraries, middleware, drivers, and development tools.

The ROS 2 workspace was organized into specialized packages for major robotic functions, including hardware interfaces, perception, navigation, control, robot description, custom messages, bringup, and behavior coordination. This package-level organization reflected a scalable software architecture in which each subsystem had a defined technical role. In addition, launch files were used to start multiple nodes in a coordinated manner, reducing manual execution errors and enabling repeatable system bringup for simulation, testing, visualization, and physical robot operation.

From a technology readiness perspective, the ROS 2 architecture progressed from subsystem level validation toward integrated system validation. Individual nodes and packages

represented early functional prototypes when tested independently, while coordinated launch execution and communication across perception, control, navigation, and hardware layers indicated a higher level of software maturity. Therefore, ROS 2 was not only used as a programming framework, but as the central middleware that allowed the PuzzleBot platform to evolve from separate software components into an integrated autonomous robotic system.

B. Computer Vision and Perception

The perception stack is organized around three distinct subsystems, each targeting a different operational need: close-range pallet identification and alignment, global landmark-based localization, and semantic delivery-site recognition.

1) *QR Code Detection and Pallet Alignment*: QR codes are printed on each pallet face and serve as the primary fiducial target for the approach and fork-insertion maneuver. The pipeline is layered into four successive stages: detection, pose estimation, filtering, and visual servoing.

a) *Detection*: The primary detector uses OpenCV's built-in `cv2.QRCodeDetector`, which runs synchronously every frame and returns the four corner points of the code as soon as a decode succeeds. This path is fast (sub-millisecond on typical hardware) but degrades under extreme viewing angles or poor lighting. When a camera intrinsic matrix K and distortion coefficients D are provided, detection is performed on the undistorted image and the resulting corner coordinates are re-projected back into the original distorted pixel space. This keeps subsequent pose estimation consistent with the raw image geometry.

b) *Pose Estimation*: Camera-to-marker pose is recovered from the four detected corner points using OpenCV's `solvePnP` (ITERATIVE solver). The physical 3D object points of the QR code, with its four corners centered at the origin in the marker plane, are matched against their 2D image-space counterparts. Because the camera is mounted inverted on the robot, a 180° pixel-coordinate flip is applied to the image points before the

PnP call, and the resulting 4×4 pose matrix CTM is stored in its post-flip convention.

The full perception result is encapsulated in a PnpResult object that also retains the matched point correspondences, enabling downstream reprojection (used when the marker is temporarily occluded and only a predicted position is available).

c) Temporal Filtering: Raw PnP output is noisy due to pixel quantization and lighting variation. A frame-rate-independent exponential moving average (EMA) filter is applied separately to the translation vector and to the rotation (stored as a unit quaternion to allow valid spherical interpolation):

where Δt is the wall-clock interval since the last accepted sample and τ is a tunable time constant (default $\tau = 0.78$ s, equivalent to $\alpha = 0.05$ at 25 Hz). Before blending, the incoming quaternion is sign-flipped if necessary to ensure the shortest-arc interpolation path is taken. A maximum-jump threshold (0.25 m by default) rejects outliers; if five consecutive samples are rejected, the filter snaps to the current measurement to recover from large scene changes.

Optical-flow corner tracking (CornerTracker, Lucas-Kanade) seeds feature points inside the last known marker quad and propagates them across frames via a homography fit while the decoder is temporarily failing, providing continuity to the control loop.

d) Visual Servoing Controller: The marker pose is transformed from the camera frame to the robot's body frame via `cam_to_car`, which compensates for the camera's lateral and forward mounting offsets by iteratively zeroing the pitch and roll residuals and applying a translation correction.

The MarkerServoing controller then computes normalized differential-drive commands $\{x, w\} \in [-1, 1]$. The angular command is a proportional controller on the horizontal pixel error between the marker's projected center and a reference pixel.

The linear command is proportional on depth error, gated by a linear authority factor that smoothly fades forward motion to zero when the gaze angle exceeds a threshold (default 20°), preventing the robot from driving forward while still largely misaligned.

A hysteresis-based goal state machine (with a configurable dwell timer, default 10 s) confirms alignment before triggering the open-loop forward insertion drive and subsequent forklift raise/lower sequence.

2) ArUco Markers for Global Localization: ArUco fiducial markers are placed at known positions in the operating environment and serve as absolute pose anchors for the navigation subsystem. Two detector configurations are supported:

- Single-marker (ArucoDetector): detects one marker by its dictionary ID and recovers a 4-DoF pose from its four corners.
- Gridboard (GridboardDetector): a 3×4 grid of markers (using `cv2.aruco.GridBoard`) provides more corners for a more robust and over-determined PnP solve. Marker sep-

aration is derived automatically from the desired physical board width.

Both configurations feed into the same PoseEstimator/PoseFilter stack described in Section III-A. The resulting camera-to-landmark transform is fused with odometry data in the navigation layer (discussed separately) to maintain a globally consistent robot pose throughout the mission.

3) Company Logo Detection for Delivery-Site Identification: To determine which storage bay to deliver a pallet to, the system identifies company logos affixed to the rack faces using a YOLO-based object detector. The network is trained to classify each logo into one of the known company categories; the bounding-box centroid and estimated distance from the camera are used to associate a detected logo with a physical bay location. This semantic labeling step runs independently of the fiducial pipeline and its result is passed to the mission planner, which selects the appropriate navigation goal

pose for deposit. The YOLO inference path is designed to run asynchronously so that it does not impact the real-time alignment controller. Fig. 2 shows the real-time classification of company logos for bay identification.



Fig. 2. Real-time delivery-site identification using YOLO-based logo detection.

C. Intelligent Robotics Pipeline

The PuzzleBot architecture can be understood as an intelligent robotic pipeline that transforms raw sensor information into physical motion and manipulation actions. Instead of treating perception, localization, navigation, control, and actuation as independent blocks, the system was designed as a sequential and interconnected chain in which the output of one subsystem becomes the input of the next one. This structure allowed the robot to process information from its environment, estimate its state, decide where to move, generate control commands, and execute actions through its mobile base and forklift mechanism.

At the first level of the pipeline, the robot acquires information from cameras, LiDAR, IMU, wheel encoders, and forklift encoder feedback. These signals provide complementary information about the robot and its environment. Cameras support QR, ArUco, and object detection; LiDAR provides obstacle and map related information; the IMU and wheel encoders support odometry and motion estimation; and the forklift encoder provides feedback about the vertical position of the lifting mechanism.

The second level corresponds to perception. In this stage, raw sensor data is transformed into meaningful information for

the robot. QR codes and ArUco markers are used as visual references for identification, localization support, and pallet alignment. Object detection contributes to recognizing relevant elements in the warehouse environment, while camera based pose estimation converts visual landmarks into geometric information that can be used by the control and navigation layers.

The third level corresponds to localization and state estimation. The robot uses odometry, inertial information, map data, and visual corrections to estimate its pose in the workspace. This pose can be represented as

$$\mathbf{x}_t = \begin{bmatrix} x_t \\ y_t \\ \theta_t \end{bmatrix}$$

where x_t and y_t represent the robot position in the map frame, and θ_t represents its heading angle. This state estimate is essential because navigation decisions depend on knowing where the robot is located relative to racks, conveyors, pallets, obstacles, and trailer zones.

The fourth level corresponds to navigation and mission planning. Once the robot estimates its current pose and receives a mission objective, the navigation layer computes a route toward the target area. This includes selecting keypoints, planning motion through the map, avoiding obstacles, and generating a trajectory that can be followed by the robot. The finite state machine coordinates this process by deciding when the robot should search for QR codes, move toward a keypoint, align with a pallet, activate the forklift, or navigate toward a trailer zone.

The fifth level corresponds to control. In this stage, planned trajectories and mission decisions are converted into executable velocity commands for the mobile base and setpoints for the forklift mechanism. A simplified relationship for the robot command can be expressed as

$$\mathbf{u}_t = \begin{bmatrix} v_t \\ \omega_t \end{bmatrix}$$

where v_t is the commanded linear velocity and ω_t is the commanded angular velocity. These commands are interpreted by the robot control system to move the differential drive base. For the forklift, the high level setpoint is converted into embedded control actions through the FPGA, PWM generation, and encoder feedback.

The final level of the pipeline is physical action. At this point, software decisions are executed through real actuators: the wheels move the robot through the workspace, and the forklift mechanism raises or lowers the payload according to the mission requirements. This completes the full robotic chain from sensing to action.

In abstract form, the complete pipeline can be represented as

$$\mathbf{a}_t = f_{act}(f_{ctrl}(f_{nav}(f_{loc}(f_{perc}(\mathbf{z}_t))))))$$

where $\mathbf{z}_t$ represents the sensor measurements, f_{perc} is the perception layer, f_{loc} is the localization layer, f_{nav} is the navigation and mission planning layer, f_{ctrl} is the control

layer, and f_{act} represents the final physical actuation. This expression summarizes the central idea of the system: sensor data is progressively transformed into motion and manipulation behavior.

This pipeline was also supported by simulation and visualization tools. The Gazebo digital twin allowed the same general architecture to be tested in a controlled warehouse environment before physical deployment, while visualization tools helped verify whether sensor data, localization, navigation, and control outputs were consistent. Therefore, the intelligent robotics pipeline was not only a conceptual organization, but the practical structure that enabled PuzzleBot to connect perception, autonomy, embedded control, and physical operation into a complete robotic system.

IV. EMBEDDED CONTROL AND HARDWARE INTEGRATION

The integration of the custom electronic subsystems and physical components is shown in Fig. 3, which illustrates the PCB design and component layout of the robot.

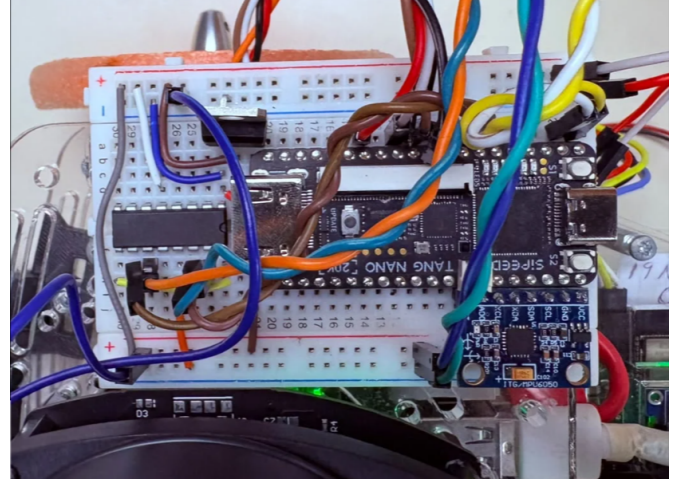


Fig. 3. Hardware integration and custom PCB design for the PuzzleBot and forklift mechanism.

A. FPGA-Based Control Subsystem

The core of the low-level control for the forklift resides in a Tang Nano 20K FPGA, which acts as a high-criticality embedded layer. This hybrid architecture allows the delegation of strict real-time tasks and direct hardware control to programmable logic, freeing the main processor (Jetson/ROS 2) from this load and guaranteeing minimum latency in command execution.

System Architecture: Softcore PicoRV32 To manage the control logic and the state machine, a PicoRV32 softcore based on the RISC-V architecture was implemented. This processor coordinates internal peripherals and executes positioning algorithms, allowing software flexibility within the determinism of the hardware. The high-level interaction between the software stack and hardware peripherals is illustrated in Fig. 4.

Control and Acquisition Components:

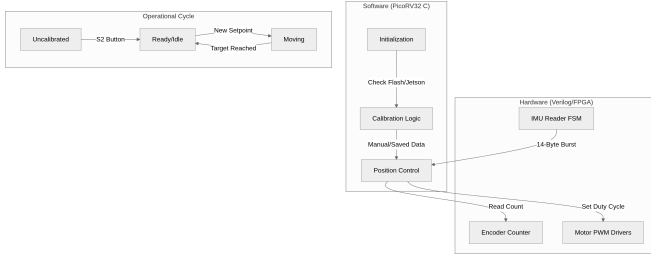


Fig. 4. System architecture showing the interaction between PicoRV32 software and FPGA hardware logic.

- **PWM Generation for Forklift:** The system features dedicated hardware PWM (Pulse Width Modulation) modules for forklift motor control. This allows fine control of speed and direction, implementing acceleration ramps and "slow zones" to prevent mechanical bounces.
- **Quadrature Encoder Reading:** A hardware quadrature decoder was integrated to process encoder signals in real-time. This ensures that no position pulse is lost, allowing ultra-precise tracking of the forklift mast state, even at high speeds.
- **Sensor Management (IMU MPU6050):** The FPGA manages IMU data acquisition via I2C, pre-processing the data so it is immediately available for the control loop. The internal state machine for handling both configuration and continuous burst reads is shown in Fig. 5.

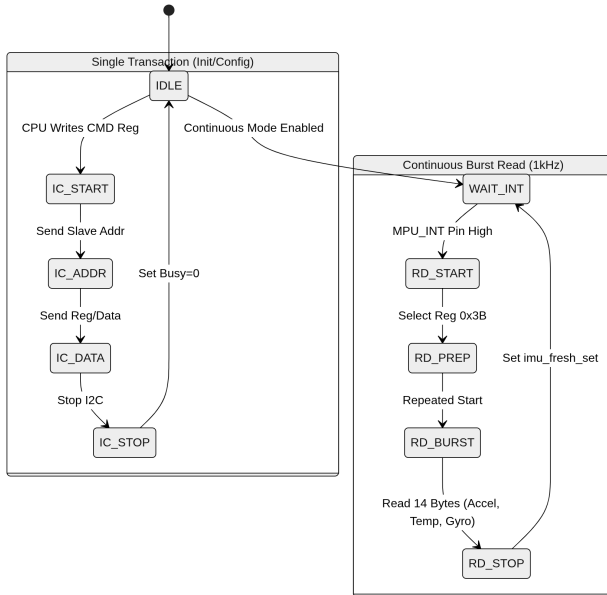


Fig. 5. I2C State machine for single configuration transactions and continuous 1kHz burst reads of the MPU6050.

Communication and Synchronization:

- **Shared Register Bank:** A register bank was designed to function as an information bridge between the hardware peripherals, the PicoRV32 softcore, and the external interface. This design allows different processes to access

encoder positions or sensor states asynchronously and safely.

- **SPI Link (Jetson ↔ FPGA):** Communication with the Jetson is performed through an optimized hardware SPI slave. Unlike a conventional serial port, this interface allows ROS 2 to read complete sensor frames and write position setpoints directly into the register bank with negligible latency, ensuring seamless integration into the robotics ecosystem.

Added Value: This implementation demonstrates that actuator control does not depend on the high-level operating system (ROS 2), but resides in a deterministic layer capable of reacting to collisions or mechanical stalls autonomously, guaranteeing hardware integrity.

B. Sensor and Actuator Interface

The interface between the control software and physical hardware was designed under a Memory-Mapped I/O scheme, centralizing data flows in the FPGA's shared register bank.

- **Inertial Measurement Unit (IMU MPU-6050):** The FPGA firmware is responsible for initializing and continuously reading the IMU through burst reads over the I2C bus. Acceleration, gyroscope, and temperature data are packaged and mapped directly into the first addresses of the register bank, staying constantly updated (with a "new data" flag) for when the Jetson decides to query them.
- **Positioning with Quadrature Encoder:** The encoder of the forklift's lifting mechanism is read by a dedicated hardware module (encoder_counter). This 32-bit counter is absolute; the Jetson or softcore can query it in real-time (addresses 0x0E-0x11 of the SPI map) without the risk of short-term overflows or loss of ticks during fast movements.
- **Forklift Motor Control (PWM Signals):** Movement commands are not sent as simple "up" or "down" instructions. The PicoRV32 manages independent PWM (pulse width modulation) registers for each motor direction. The state machine in C (forklift.c) regulates the duty cycle to implement movements at maximum power and slow approach zones near the target, reducing mechanical inertia.
- **Dynamic Position Calibration:** Due to the mechanical nature of the forklift, an advanced calibration system was implemented. The firmware allows for dynamically defining the lower ("bottom") and upper ("top") limits of the mechanism, translating this into a normalized setpoint (from 0.0 to 1.0). Additionally, it features "stall detection" routines that detect mechanical blockages or loss of progress to automatically invert motor polarity if the encoder or wiring polarity changes. The operational state machine for the forklift mechanism is shown in Fig. 6.
- **SPI Communication Channel:** All subsystem interaction with the main computer (Jetson) is unified on an SPI bus. Through a structured frame, the Jetson receives

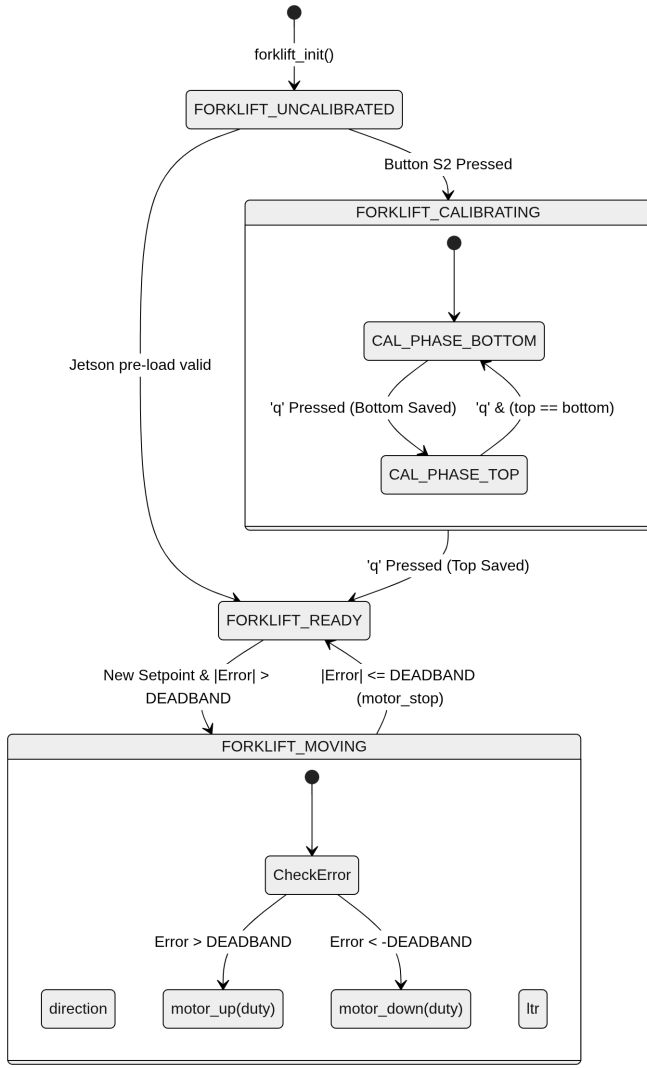


Fig. 6. Operational state machine for the forklift mechanism, including calibration and error-based position control.

all telemetry data (IMU, Encoder, Status) in a single transaction and can send back the desired setpoint and previously saved calibration parameters, allowing a high-frequency closed-loop control flow.

C. Embedded Validation

During the development of the control subsystem on the FPGA, a modular design approach was adopted that allowed for iterative testing of hardware components and softcore state machines.

Successful implementation and validation of uninterrupted sensor reading (IMU and encoder), PWM signal generation for the motor H-bridge, and the bidirectional SPI communication interface were achieved. The internal logic in C demonstrated robustness in detecting collisions or mechanical stalls and the ability to converge stably toward a given setpoint, reacting appropriately in the local control loop. This phase was critical for the STE0103C subcompetency, as it validated that

the embedded system could maintain a stable 1kHz control frequency regardless of the computational load on the main Jetson processor.

Despite these advances at the physical and firmware layer levels, the validation of the system as a whole is an ongoing process. The embedded subsystem provided a solid foundation for highly deterministic and reliable actuator control; however, complete integration between the low-level firmware, the final physical calibration on the robot chassis, and the mission behaviors planned by ROS 2 requires exhaustive end-to-end validation. This future work will ensure that mechanical tolerances and full-load robot dynamics perfectly match the control loop parameters implemented in the FPGA.

V. AUTONOMOUS NAVIGATION AND ROBOTIC BEHAVIOR

The autonomous navigation system is a core component of the PuzzleBot architecture, integrating localization, path planning, and mission execution.

A. Mobile Robot Architecture

1) *Kinematic Derivation*: The PuzzleBot is a differentially-steered mobile robot, controlled by individually steering two wheels. Moving them in the same direction drives the robot forward, while different speeds cause it to rotate around its center point. To derive the robot's motion in the global coordinate frame (x, y, θ) from wheel angular velocities (ω_R, ω_L) , we first define the linear v and angular ω velocities:

$$v = \frac{r}{2}(\omega_R + \omega_L), \quad \omega = \frac{r}{L}(\omega_R - \omega_L)$$

where r is the wheel radius and L is the track width. In the global frame, the kinematics are:

$$\dot{x} = v \cos(\theta), \quad \dot{y} = v \sin(\theta), \quad \dot{\theta} = \omega$$

Substituting the wheel velocities into these equations, we obtain the state transition matrix [9]:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \frac{r}{2} \cos(\theta) & \frac{r}{2} \cos(\theta) \\ \frac{r}{2} \sin(\theta) & \frac{r}{2} \sin(\theta) \\ \frac{r}{L} & -\frac{r}{L} \end{bmatrix} \begin{bmatrix} \omega_R \\ \omega_L \end{bmatrix}$$

2) *Implementation in ROS 2*: We utilize these equations to publish linear velocity (v) commands to the `/cmd_vel` topic. Encoder data is processed through these equations to track the robot's odometry within a dedicated node in the ROS 2 workspace.

For trajectory control, the robot operates with a pre-scanned world map. We use a weighted A* algorithm to identify optimal poses between the current position and the target. To mitigate odometry drift, a Kalman Filter fuses LiDAR and IMU data against the robot's internal model. A pose controller, utilizing a Pure Pursuit algorithm, dynamically assigns poses along the path. For obstacle avoidance, the controller can activate a Bug 2 Algorithm, which circles obstacles until a clear path to the destination is established. The switching logic between goal-seeking and wall-following modes is illustrated in Fig. 7.

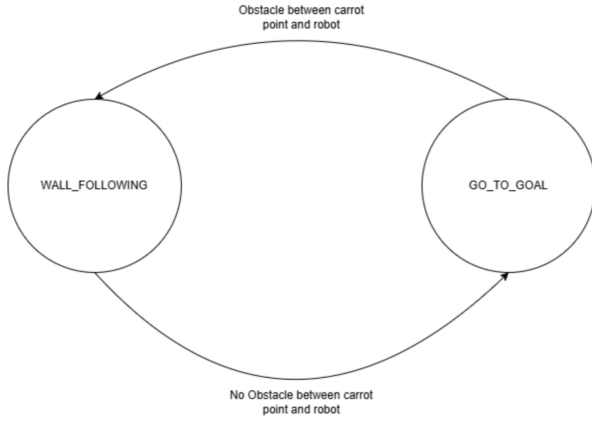


Fig. 7. Bug 2 Algorithm state machine showing transitions between wall-following and goal-seeking behaviors based on obstacle detection.

B. Localization and Odometry

The localization system for the Puzzlebot is designed as a multi-tier architecture, prioritizing robust motion tracking through sensor fusion and high-performance scan matching. While the base odometry is handled by an external Extended Kalman Filter (EKF), this package implements advanced Monte Carlo Localization (MCL) and a custom SLAM pipeline.

1) *Odometry and Motion Prediction*: To provide a stable, high-frequency pose candidate for the localization pipeline, an Extended Kalman Filter (EKF) fuses wheel encoder velocities with raw IMU gyroscope data. This estimator is modeled with a 5-dimensional state vector to account for both robot kinematics and sensor imperfections:

$$x = [x \quad y \quad \theta \quad \omega \quad \text{bias}]^T$$

- (x, y, θ) : The 2D world-frame pose.
- ω : The estimated "true" angular velocity.
- bias : The estimated gyroscope bias (drift).

Motion Model & Prediction:

The prediction step occurs at approximately 60Hz, triggered by encoder updates. It utilizes a constant angular velocity model between correction steps:

$$f(x, u) = \begin{bmatrix} x + v \cos(\theta) \Delta t \\ y + v \sin(\theta) \Delta t \\ \theta + \omega \Delta t \\ \omega \\ \text{bias} \end{bmatrix}$$

Observation Strategy:

The filter employs a decoupled, asynchronous correction strategy to maintain stability across different surface frictions:

- Encoder Correction ($H_{enc} = [0, 0, 0, 1, 0]$): Derives angular velocity from differential drive kinematics ($\omega_{enc} = \frac{v_r - v_l}{L}$). The filter is tuned with low measurement noise ($R_{enc} = 10^{-3}$) to trust encoders for short-term rotation.
- IMU Correction ($H_{imu} = [0, 0, 0, 1, 1]$): The IMU measures the sum of the true angular velocity and the bias.

By comparing this against the encoder-derived rotation, the EKF dynamically estimates and compensates for the gyroscope's thermal drift (modeled as a random walk in Q with 10^{-5} variance).

2) *Monte Carlo Localization (MCL)*: For localization against a known static map, a GPU-accelerated Particle Filter is employed.

- High Performance: Utilizing custom CUDA kernels via CuPy, the filter can evaluate thousands of particles in real-time. Each particle is scored by projecting lidar beams onto the occupancy grid.
- Kidnapped Robot Solution: To handle cases where the robot is manually moved or loses tracking, the implementation uses Augmented MCL. By monitoring the short-term (w_{fast}) and long-term (w_{slow}) likelihood averages, the system detects tracking failures and injects random particles to facilitate global relocalization.
- Adaptive Noise: The motion model applies dynamic noise that scales with the robot's linear and angular displacement, ensuring that drift is only introduced when the robot is in motion.

3) CoreSLAM (Simultaneous Localization and Mapping):

When a map is not provided, the robot uses a custom CoreSLAM implementation for incremental mapping. This pipeline uses a hybrid three-stage refinement process:

- 1) ICP Scan-to-Scan: Before map matching, an Iterative Closest Point (ICP) algorithm aligns the current lidar scan with the previous one. This reduces odometry drift in the "short term" before the map is updated.
- 2) Stochastic Search: A cloud of random pose candidates is sampled around the ICP-refined pose and scored against a Likelihood Field.
- 3) Gauss-Newton Refinement: The best candidate is further optimized using a gradient-based Gauss-Newton method. By calculating the gradient of the likelihood field using bilinear interpolation, the system achieves sub-pixel (centimetric) pose accuracy. The resulting environment mapping and real-time pose estimation is demonstrated in Fig. 8.

4) *Sensor Fusion and Markers*: The localization is further stabilized by:

- Likelihood Fields: Instead of raw occupancy grids, a distance-transformed likelihood field is used for scoring, providing a smooth gradient for optimization.
- Visual Landmarks (ArUco): The SLAM node integrates absolute pose corrections from ArUco markers. These measurements are "soft-injected" into the stochastic search, acting as an anchor to prevent long-term map deformation.

5) *Parameter Optimization via Genetic Algorithm*: To ensure peak performance, the SLAM hyperparameters were tuned using a Genetic Algorithm (GA). The GA simulates the SLAM process on recorded datasets and optimizes for minimal Map Entropy, resulting in sharp, consistent maps even in feature-poor environments. The optimization identified a set of

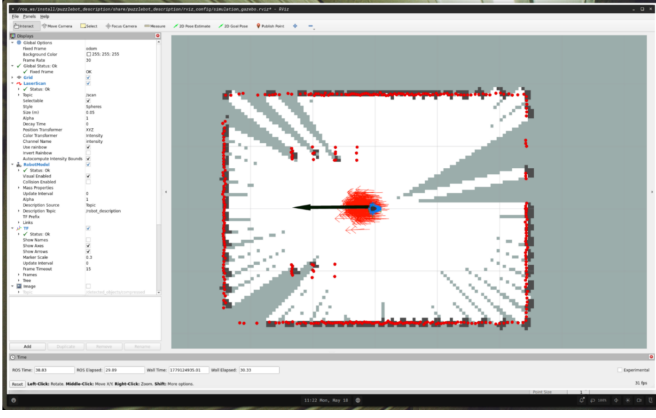


Fig. 8. Real-time SLAM execution showing the generated occupancy grid and robot pose estimation.

high-performing parameters that minimized map uncertainty (entropy = 0.0295), as detailed in Table II.

TABLE II
OPTIMIZED SLAM PARAMETERS FOUND VIA GENETIC ALGORITHM

Parameter	Optimized Value
n_candidates	444
sigma_xy	0.1642
sigma_theta	0.0723
hit_alpha	0.9200
free_alpha	0.4000
lf_sigma	3.4876
icp_dist_thresh	0.1227
gn_iters	4

C. Navigation and Path Planning

The navigation layer was responsible for transforming mission objectives into executable robot motion. In the PuzzleBot architecture, navigation was not treated as a single process, but as a combination of map based planning, waypoint selection, trajectory generation, path following, and local maneuver execution. This separation was important because planning and control solve different problems: planning determines where the robot should move, while control determines how the robot physically follows that motion.

The planning stage used the map of the warehouse environment to compute feasible routes toward relevant mission zones, such as racks, conveyors, pallet areas, and trailer bays. The robot operated over a structured environment that included static elements, navigation corridors, visual landmarks, and task specific target zones. From this information, the navigation layer selected keypoints and generated a route that minimized unnecessary displacement and rotation effort.

A graph based route representation can be expressed as

$$G = (V, E)$$

where V represents the set of navigation keypoints and E represents the feasible connections between them. A route can then be defined as an ordered sequence of waypoints,

$$R = \{p_1, p_2, \dots, p_n\}$$

where each waypoint $p_i = (x_i, y_i, \theta_i)$ represents a desired position and orientation in the workspace. This structure allowed the mission logic to calculate a route, select the next target, and update the active goal during execution.

For path planning, the system followed a cost based navigation strategy. A costmap represented the workspace by assigning traversal costs to different areas depending on obstacle proximity, free space, and navigability. In this context, A* was used as a search method to find a feasible path between the robot position and the target waypoint. The basic A* evaluation function can be represented as

$$f(n) = g(n) + h(n)$$

where $g(n)$ is the accumulated cost from the start node to the current node, and $h(n)$ is a heuristic estimate of the remaining cost from the current node to the goal. This allowed the robot to plan paths that considered both traveled distance and estimated distance to the target.

Once a path was planned, the control stage was responsible for tracking it. The Pure Pursuit controller was used to convert the planned trajectory into velocity commands for the differential drive base. Instead of commanding the robot to move directly toward the final goal, Pure Pursuit selects a lookahead point on the path and generates motion commands that steer the robot toward that point. This allows smoother trajectory following and reduces abrupt corrections during navigation.

The geometric relationship used by Pure Pursuit can be summarized through the curvature command,

$$\kappa = \frac{2y_L}{L_d^2}$$

where κ is the required path curvature, y_L is the lateral displacement of the lookahead point in the robot frame, and L_d is the lookahead distance. The angular velocity command can then be approximated as

$$\omega = v\kappa$$

where v is the commanded linear velocity and ω is the angular velocity required to follow the path. This controller connected the planned trajectory with the actual motion of the robot base.

In addition to planned navigation, the system also required open loop control for short and precise maneuvers. These maneuvers were useful when sensor based navigation was not the most practical option, such as moving the robot forward into a pallet after alignment, reversing out of a loading area, or executing small controlled motions during pallet handling. Open loop control was not used as the main navigation method, but as a complementary tool for task specific movements where the robot needed predictable short distance behavior.

The navigation layer was also coordinated by the mission finite state machine. The FSM determined when the robot should calculate a route, pop the next waypoint, move toward a keypoint, sweep for QR detection, align with a pallet, move the forks, or navigate toward the trailer zone. This allowed navigation to support the complete task sequence rather than operating as an isolated path following module.

Overall, the navigation and path planning subsystem connected spatial reasoning with physical motion. The planner selected feasible routes through the environment, the controller translated those routes into executable velocity commands, and the mission logic decided when each navigation behavior should be activated. This separation between planning, control, and mission coordination made the PuzzleBot architecture more modular and better suited for autonomous warehouse operation.

D. Autonomous Mission Behavior

The autonomous mission behavior was coordinated through a finite state machine that connected perception, navigation, manipulation, and task execution into a sequential mission flow. This layer was essential because the robot did not only need to move from one point to another; it had to understand the current mission, select target zones, search for visual information, align with a pallet, actuate the forklift, transport the load, identify the correct trailer area, and complete the delivery sequence.

The mission began in a waiting state, where the robot remained idle until a valid mission command was received. Once the mission was activated, the system calculated a route using the current robot pose and the available keypoints in the warehouse environment. This route was organized as an ordered sequence of targets that allowed the robot to move through relevant zones such as racks, conveyors, pallet locations, and trailer areas. After the route was calculated, the next active waypoint was selected and passed to the navigation layer.

The finite state machine can be represented as

$$M = (S, I, O, \delta, s_0)$$

where S is the set of mission states, I is the set of input conditions, O is the set of output actions, δ is the transition function, and s_0 is the initial state. In this project, the transition function determined when the robot should remain idle, calculate a route, navigate to a keypoint, search for a QR code, align with a pallet, command the forklift, move toward the trailer zone, or finish the mission.

During navigation, the robot moved toward selected keypoints using the planned route and the path following controller. Once the robot reached a target zone, it executed a sweep behavior to search for QR codes or visual markers. This allowed the robot to identify relevant objects in the environment instead of blindly assuming that the target was already visible. The QR reading stage provided task specific information that could be used to continue the mission se-

quence, associate the detected pallet with a target class, or trigger the next manipulation behavior.

After a valid pallet reference was detected, the FSM transitioned into the alignment and manipulation states.

The QR pallet alignment stage used the visual pose of the marker to square the robot with the payload. Once the robot was aligned, the forklift controller moved the forks to the required pallet height. The robot then executed a controlled forward motion to insert the forks into the pallet, followed by a lifting action to secure the payload. This sequence showed that navigation and manipulation were not independent behaviors; they were coordinated through mission level logic.

Once the pallet was secured, the robot moved away from the pickup area and navigated toward the trailer zone. In this stage, the mission behavior changed from pallet acquisition to payload delivery. The system searched for the correct trailer reference, using visual detection to identify the target bay. After the correct destination was selected, the robot moved into the trailer area, lowered the forks to release the pallet, and reversed out from underneath the load.

The mission behavior also included waiting and recovery logic. If a required condition was not satisfied, such as a missing QR detection, unavailable target, incomplete alignment, or unexpected navigation behavior, the FSM could remain in the current state, repeat a search action, wait for valid input, or allow operator intervention through the interaction layer. This was important because real robotic systems must handle uncertainty rather than assuming that every perception or navigation step will succeed on the first attempt.

Table III summarizes the main states involved in the autonomous mission behavior.

Overall, the finite state machine allowed the PuzzleBot to transform independent modules into a coordinated autonomous behavior. Perception provided object and marker information, navigation moved the robot through the warehouse, control executed precise movements, and the forklift subsystem performed the manipulation task. By sequencing these capabilities through mission states, the system achieved autonomy at the task level rather than only at the motion level.

VI. INTELLIGENT INTERFACES AND HUMAN-ROBOT INTERACTION

A. Voice User Interface

The PuzzleBot system incorporated an offline voice user interface as an additional interaction layer for command execution. Instead of depending on cloud based speech recognition services, the voice system was designed to operate locally. This improved privacy, reduced dependency on external network connectivity, and decreased latency during basic command interaction. This design decision was relevant for mobile robotics applications because wireless connectivity may become unstable during testing, while command execution must remain available even without internet access.

The voice interface followed an isolated word recognition approach. The system first listened for the wake word “robot” before accepting movement commands, reducing the

TABLE III
AUTONOMOUS MISSION STATE SEQUENCE

Mission state	Main purpose	Related subsystem
Wait mission	Remain idle until a valid mission is received	Mission control
Calculate route	Generate an ordered route from the current pose to mission targets	Route planning
Pop next	Select the next active waypoint from the route	Mission coordination
Go to keypoint	Navigate toward the selected target position	Navigation and control
Sweep	Rotate to search for QR or visual references	Perception
Align with pallet	Use visual pose information to square the robot with the payload	QR pose estimation and alignment
Move forks to pallet height	Set the forklift height required for pallet pickup	Forklift control
Move robot into pallet	Execute a controlled forward movement into the pallet	Open loop control
Move forks to secure pallet	Lift the pallet to secure the load	Forklift control
Move away from area	Reverse out of the pickup area with the pallet secured	Open loop control
Go to trailers zone	Navigate toward the trailer staging area	Navigation
Wait YOLO match	Detect and identify the correct trailer reference	Object detection
Move to trailer	Drive into the selected trailer bay	Navigation and control
Move forks to floor height	Lower the pallet for delivery	Forklift control
Move robot outside pallet	Reverse out from under the delivered pallet	Open loop control
Mission complete	Stop or return to idle after successful delivery	Mission control

probability of accidental command execution. After the wake word was detected, the user could issue discrete movement commands such as forward, backward, left, right, and stop. These commands were mapped into robot motion actions and forwarded to the ROS 2 ecosystem through a bridge layer.

The audio processing pipeline was structured around microphone acquisition, voice activity detection, feature extraction, temporal classification, command decision logic, and ROS 2 command publishing. Voice activity detection was used to isolate valid speech segments from the microphone stream. A simple energy based criterion can be represented as:

$$E = \frac{1}{N} \sum_n x[n]^2$$

where $x[n]$ is the sampled audio signal and N is the number of samples in the analysis window. If the signal energy exceeds a selected threshold, the audio segment is considered a valid speech candidate and is passed to the feature extraction stage.

Once a valid speech segment was detected, Mel frequency cepstral coefficients were extracted to represent the spectral structure of the spoken command. These coefficients provided a compact representation of the audio signal and were used as observations for temporal command classification. A Hidden Markov Model was then used to evaluate the sequence of MFCC derived observations against trained word models, allowing the recognizer to account for temporal variations in speech commands. In general terms, the recognized command

can be expressed as:

$$w^* = \arg \max_w P(O \mid \lambda_w)$$

where O is the observation sequence extracted from the spoken command, λ_w is the Hidden Markov Model associated with a specific word, and w^* is the word model with the highest likelihood.

After recognition, the command was translated into a ROS 2 compatible velocity payload. The bridge published the corresponding command to the robot motion stack, allowing the voice interface to affect the same control architecture used by the rest of the system. A basic velocity command can be represented as:

$$u = [v, \omega]^T$$

where v is the linear velocity and ω is the angular velocity. For example, the forward command produces a positive linear velocity and approximately zero angular velocity, while left and right commands modify the angular velocity to rotate the robot. In this way, the voice interface was not implemented as an isolated demonstration, but as a functional interaction layer connected to the complete PuzzleBot architecture.

The main value of the voice interface was not only speech recognition, but its integration with the robot control stack. By using an offline recognition pipeline, a wake word finite state machine, and a ROS 2 bridge, the system enabled a lightweight and local human input channel for robot motion commands.

B. Web Interface and HMI

The project also incorporated a web based human machine interface for robot supervision, monitoring, and control. This interface was designed as an operator facing layer that made the internal state of the robot more accessible during development and testing. Rather than treating the web system as a simple webpage, it was structured as an HMI capable of connecting user interaction with the robotic software stack.

The web architecture was organized around a frontend, a backend, and a robot communication layer. The frontend provided the visual interface for the operator, while the backend acted as an intermediate layer between the web application and the robot side software. Through WebSocket communication, ROS bridge mechanisms, or dedicated robot nodes, the interface could exchange information with the robot and support monitoring or command transmission. The live dashboard for system monitoring and control is shown in Fig. 9.



Fig. 9. The PuzzleBot web-based HMI showing real-time telemetry, camera feeds, and manual control overrides.

This HMI layer was useful because autonomous robots require continuous observability during testing. The operator must be able to inspect whether the robot is receiving commands, publishing odometry, updating its mission state, detecting markers, or responding correctly to navigation instructions. A web based interface provides this visibility in a more accessible format than terminal outputs alone.

For a differential drive robot, the relationship between the robot velocity command and the wheel angular velocities can be expressed as

$$v = \frac{r}{2}(\omega_R + \omega_L)$$

$$\omega = \frac{r}{L}(\omega_R - \omega_L)$$

where r is the wheel radius, L is the distance between wheels, and ω_R and ω_L are the angular velocities of the right and left wheels. Therefore, when the HMI or voice interface sends a high level velocity command, the control system can transform it into wheel level actuation using

$$\omega_R = \frac{v + \frac{L}{2}\omega}{r}$$

$$\omega_L = \frac{v - \frac{L}{2}\omega}{r}$$

These equations show that the interface layer does not directly control the motors. Instead, it sends high level commands that are interpreted by the robot control architecture.

The web interface also supported the broader integration strategy of the project. While ROS 2 nodes handled internal robotic computation, the HMI translated relevant system information into a format that could be interpreted by a human operator. This separation allowed the robot to maintain its autonomous software architecture while still providing external supervision and intervention capabilities.

C. Visualization and Debugging Tools

Visualization and debugging tools were essential for integrating the complete PuzzleBot architecture. Since the system combined perception, navigation, embedded control, voice commands, web interaction, and physical actuation, terminal logs alone were not sufficient to understand the behavior of the robot. Tools such as Foxglove, RViz, and perception dashboards provided visual access to the internal state of the system and supported faster diagnosis during physical testing.

Foxglove was used as a robotics visualization environment for inspecting ROS 2 topics, sensor data, command messages, odometry information, and system behavior during execution. This was important because many integration failures in robotics are not caused by a single incorrect algorithm, but by

inconsistencies between published topics, coordinate frames, timing, sensor data, and actuator commands. By visualizing the communication flow, the team could verify whether the robot was actually receiving and publishing the expected information.

RViz complemented this process by supporting the visualization of maps, transforms, robot models, localization outputs, sensor data, and navigation related information. In autonomous mobile robotics, visualizing coordinate frames, robot pose, map alignment, and trajectory behavior is critical for validating whether the robot's internal representation matches the physical environment. A robot pose in the plane can be represented as

$$\mathbf{x} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix}$$

where x and y represent the robot position in the map frame, and θ represents its heading angle. Visualizing this state helps verify whether localization, odometry, and navigation outputs are consistent with the physical robot.

Perception dashboards were also valuable because they provided direct feedback from the visual processing layer. Camera based tools for QR detection, fiducial marker recognition, and object detection required visual confirmation during testing. These dashboards helped verify whether the perception system was detecting the correct targets, producing useful pose

information, and providing data that could be used by the navigation or alignment logic.

Overall, the visualization layer increased the observability of the robot. This made it possible to move from black box testing, where the robot either works or fails, toward a more structured debugging process in which each subsystem could be inspected independently and then validated as part of the complete system.

D. Communication and Teleoperation

Communication and teleoperation tools were implemented as supporting interfaces for testing, calibration, debugging, simulation, and recovery. Although the final objective of the project was autonomous operation, teleoperation remained necessary during development because it allowed the team to manually move the robot, test sensor responses, validate actuator behavior, and recover the platform when autonomous execution failed.

A relevant part of the control architecture was the implementation of a priority multiplexer for motion commands. Since the robot could receive commands from different sources, such as joystick, keyboard, voice interface, web based UI, autonomous controllers, and simulation tools, the system required a mechanism to decide which command source should control the robot at each moment. The priority mux acted as a safety and coordination layer between the available controllers and the final PuzzleBot plus forklift platform, whether running in simulation or on the physical robot.

This design prevented multiple interfaces from commanding the robot simultaneously without control arbitration. For example, during autonomous operation, the navigation controller could generate motion commands, while during debugging or emergency recovery, teleoperation commands from a joystick, keyboard, or UI could take priority. Similarly, voice commands could be integrated as a high level interaction channel without bypassing the command hierarchy. This made the communication layer more robust because it converted several independent input sources into a controlled and prioritized command flow.

The communication layer also included auxiliary scripts and tools for sending commands, receiving robot state information, and connecting external user interfaces with the robot. These tools supported different stages of integration, from early manual control to more advanced communication between the web interface, ROS 2 nodes, Gazebo simulation, and hardware subsystems. Teleoperation through keyboard, joystick, web controls, or client server utilities allowed the team to validate basic mobility before relying completely on autonomous navigation.

Simulation was also used as part of the validation workflow. A digital twin of the robot and its working environment was created in Gazebo, reproducing the PuzzleBot with the forklift mechanism and the warehouse style operating area. The simulated environment included racks, conveyors, pallets, QR based objects, ArUco markers, LiDAR, cameras, physical dynamics, inertias, joint limits, and ROS 2 control interfaces.

This allowed the team to test software behavior in a controlled environment before executing the same logic on the physical robot.

The digital twin was important because it maintained a software architecture equivalent to the real robot. This means that perception, navigation, control, teleoperation, and visualization tools could be evaluated in simulation using the same general ROS 2 pipeline expected on the physical platform. As a result, Gazebo reduced integration risk by allowing the team to test autonomous routines, command routing, sensor behavior, and mission logic before exposing the real robot to possible mechanical or navigation failures.

Teleoperation was also important for calibration. Before testing high level behaviors, the robot needed to be positioned, aligned, and verified under controlled conditions. Manual control made it possible to test camera placement, LiDAR visibility, odometry response, forklift movement, command mux behavior, and communication stability without depending entirely on autonomous routines. This reduced the risk of damaging the robot during early experiments and helped isolate problems between software, hardware, simulation, and mechanical subsystems.

In addition, teleoperation acted as a fallback mechanism. When a perception, navigation, localization, or embedded control component produced unexpected behavior, the operator could regain control of the robot and safely stop or reposition it. This is an important characteristic of intelligent robotic interfaces: the

user should not only be able to command the robot, but also supervise, interrupt, debug, simulate, and recover the system when necessary.

The responsiveness of the interaction layer can be described using the total interface latency:

$$T_{total} = T_{input} + T_{processing} + T_{communication} + T_{execution}$$

where T_{input} is the time required to capture the user input, $T_{processing}$ is the time required to interpret the command, $T_{communication}$ is the time required to transmit it into the robot architecture, and $T_{execution}$ is the time required for the robot control layer to act on it. Minimizing this latency is important because delayed command execution can reduce safety, controllability, and operator confidence.

Table IV summarizes the main intelligent interaction, supervision, and validation components implemented in the PuzzleBot architecture.

The intelligent interface layer strengthened the complete PuzzleBot system by connecting autonomous behavior with human supervision, command arbitration, and simulation based validation. Voice commands, web monitoring, visualization dashboards, Gazebo simulation, priority based command routing, and teleoperation tools allowed the robot to be operated, inspected, debugged, and recovered through multiple interaction channels. This made the system more robust during integration because the user could interact with the robot at different levels: high level command execution, real time

TABLE IV
INTELLIGENT INTERACTION, SUPERVISION, AND VALIDATION COMPONENTS

Interface component	Main function	Contribution to the robotic system
Offline voice interface	Recognizes wake word and movement commands	Enables local human robot command interaction without cloud dependency
ROS 2 voice bridge	Publishes recognized commands into the robot stack	Connects speech recognition with robot motion control through /cmd_vel
Priority mux	Selects the active command source according to priority	Prevents conflicting commands from joystick, keyboard, voice, UI, autonomous controllers, and simulation tools
Web based HMI	Provides monitoring and operator interaction	Improves supervision, control, and accessibility of robot state
Real time supervision UI	Displays map, robot pose, velocity, voice commands, and lifter position	Makes autonomous behavior observable, auditable, and interruptible
Gazebo digital twin	Simulates the PuzzleBot, forklift, sensors, and warehouse environment	Enables validation of navigation, perception, control, and mission logic before physical testing
Foxglove	Visualizes ROS 2 topics and runtime data	Supports debugging of communication, commands, and sensor outputs
RViz	Visualizes maps, transforms, robot pose, and navigation data	Supports validation of localization and autonomous navigation
Perception dashboard	Displays camera and detection outputs	Helps verify QR, fiducial marker, and object detection behavior
Teleoperation tools	Enables manual robot control	Supports testing, calibration, and recovery during integration

monitoring, visual debugging, simulation testing, prioritized manual control, and direct intervention when necessary.

VII. INTEGRATION METHODOLOGY

A. Modular Development

The PuzzleBot system was developed using a modular integration methodology in which each subsystem was designed with a specific technical responsibility while maintaining shared interfaces with the rest of the architecture. This approach was necessary because the robot combined multiple engineering domains, including perception, localization, navigation, embedded control, simulation, teleoperation, web supervision, voice interaction, and physical actuation. Developing all components as a single monolithic system would have made debugging, testing, and integration significantly more difficult.

The project was divided into specialized development streams. The autonomy stream focused on navigation, localization, path following, mission behavior, and robot state estimation. The perception stream processed camera and sensor information through QR detection, ArUco based references, object recognition, and pose estimation. The embedded stream handled low level actuator control, FPGA based motor control, PWM generation, encoder feedback, and SPI communication. The interaction stream provided external access to the robot through the web HMI, offline voice commands, visualization tools, and teleoperation mechanisms. Finally, the simulation stream reproduced the robot and its working environment in Gazebo to validate behavior before physical deployment.

This modular organization was supported by ROS 2 communication interfaces. Topics were used for continuous data exchange, such as velocity commands, odometry, sensor data, visual detections, and robot state information. Services or command level messages were used for discrete interactions, such as starting missions, requesting specific actions, or sending actuator setpoints. Custom messages and shared data structures allowed different subsystems to exchange information without depending directly on each other's internal implementation.

The system was also organized into functional ROS 2 packages and launch configurations. This made it possible to start groups of nodes in a coordinated manner, reduce manual execution errors, and test different system configurations for simulation, visualization, teleoperation, and physical operation. The use of Docker and Docker Compose contributed to reproducibility by reducing dependency conflicts and allowing the software environment to remain consistent across different development machines.

Integration was performed progressively. First, individual subsystems were validated independently, such as camera perception, voice recognition, forklift actuation, web monitoring, localization, and teleoperation.

Then, pairs of subsystems were connected through shared interfaces, for example perception with navigation, voice commands with motion control, web supervision with robot state data, and FPGA control with actuator feedback. Finally, the system was tested as a complete robotic pipeline in which sensor information, mission logic, command arbitration, embedded actuation, and user supervision interacted within the same architecture.

This methodology allowed the team to isolate errors more efficiently. When a failure occurred, the system could be inspected at the level of sensor input, message publication, state estimation, navigation command, control output, embedded response, or physical actuation. As a result, modular development was not only an organizational strategy, but also a debugging and validation strategy for the complete robot.

B. Hardware–Software Integration

The hardware software integration process connected the ROS 2 robotic architecture with the physical components of the PuzzleBot platform. This included the mobile base, cameras, LiDAR, IMU, wheel encoders, forklift mechanism, DC motor, motor driver, FPGA controller, Jetson computer, web interface, voice interface, and visualization tools. The main challenge was that the software architecture operated as a distributed system, while the physical robot responded according to real electrical, mechanical, timing, and sensor constraints.

At the high level, ROS 2 coordinated perception, navigation, mission behavior, teleoperation, and supervision. At the low level, the embedded subsystem translated motion or mechanism commands into electrical signals for actuator control. This separation was important because high level robotic decisions required computation, mapping, perception, and planning, while the forklift mechanism required deterministic actuator control, PWM generation, encoder feedback, and communication with the main computing layer.

The FPGA acted as an embedded control layer between the robot’s high level software and the forklift actuator. The Jetson generated commands or setpoints based on the robot mission, and these were transmitted to the FPGA through SPI. The FPGA then interpreted the command, generated the required PWM signal, controlled motor direction through the motor driver, and processed encoder feedback to estimate the lifter position. This architecture allowed the robot to separate decision making from low level motor execution.

Sensor integration was also essential. Cameras provided visual information for QR detection, ArUco references, object detection, and pallet alignment. LiDAR supported mapping, obstacle detection, and navigation. The IMU and wheel encoders contributed to odometry and motion estimation. The forklift encoder provided feedback for the vertical position of the lifter mechanism. These sensor streams had to be synchronized conceptually through the robot architecture so that perception, localization, navigation, and actuation could operate coherently.

The integration also required connecting user interfaces with physical robot behavior. The voice interface transformed spoken commands into ROS 2 compatible velocity commands. The web HMI displayed robot state information such as map position, velocity, voice commands, and lifter position. Foxglove and RViz provided runtime visualization of topics, transforms, odometry, maps, and sensor data. Teleoperation tools allowed manual intervention during calibration, debugging, and recovery. A priority mux was used to arbitrate

between possible command sources, such as joystick, keyboard, voice, UI, autonomous navigation, and simulation, preventing conflicting control commands from reaching the robot simultaneously.

Gazebo was used as a simulation environment to reduce physical integration risk. A digital twin of the robot and warehouse environment allowed the team to test the PuzzleBot, forklift mechanism, sensors, racks, conveyors, pallets, QR objects, ArUco markers, LiDAR, cameras, dynamics, inertias, and control interfaces before deploying behavior on the real platform. This was important because the same general robotic architecture could be validated in simulation and then transferred to the physical robot with fewer unknowns.

The central difficulty of hardware software integration was the interaction between real time physical behavior and distributed software execution. A command that works correctly at the software level may still fail if the actuator lacks sufficient torque, the sensor is misaligned, the power supply is unstable, the communication link drops data, the localization estimate drifts, or the mechanical structure introduces friction. For this reason, integration required continuous validation between simulation, visualization, embedded control, and physical testing.

Overall, the hardware software integration process transformed PuzzleBot from a collection of individual subsystems into a coordinated robotic platform. ROS 2 provided communication and coordination, sensors provided environmental and state information, the embedded layer controlled physical actuation, and the interaction tools allowed the operator to supervise, debug, and recover the robot. This integration was the core requirement for achieving autonomous behavior in a real mobile robotic system.

C. Testing Strategy

Testing is organized in ascending order of integration complexity, from isolated subsystem validation up to the full end-to-end mission. Each stage must pass before the next is attempted.

1) *Isolated Subsystem Tests*: Each subsystem is verified in isolation before being connected to the rest of the stack. Table V details the specific test cases and success criteria for each hardware and low-level software component. Specifically, the FPGA-based motor control and sensor acquisition logic were verified against the state machines described in Fig. 4 and Fig. 5.

2) *Perception Tests*: The visual processing pipeline was validated under varying lighting and distance conditions. Table VI summarizes the objectives and expected evidence for the QR, ArUco, and YOLO-based detection systems.

3) *Navigation Tests*: Autonomous mobility was tested by evaluating odometry precision, obstacle avoidance behavior, and map consistency. The specific tests performed are listed in Table VII.

4) *Forklift Mechanism Tests*: The embedded control logic for the lifting mechanism was verified for positional accuracy

TABLE V
ISOLATED SUBSYSTEM TESTS

Subsystem	What is tested	Pass criterion
Differential drive	Open-loop velocity commands via pb_bridge.py	Robot moves at expected speed/direction
FPGA forklift driver	PWM generation and encoder readback over SPI	Fork reaches commanded position within tolerance
ESP32 PCB	Motor driver outputs, current sensing, E-stop	No faults under nominal load
Jetson $\leftrightarrow$ FPGA SPI link	Round-trip latency and data integrity	<5 ms latency, zero corrupt frames over 1000 transactions
ROS 2 topic bridge (rosbridge)	Publish/subscribe for /cmd_vel_safe, /forklift_controller/real_cmd, /fpga_interface/forklift_encoder	Messages received at expected rate with correct values

TABLE VI
PERCEPTION TESTS

Test	Objective	Evidence
QR code detection range	Confirm reliable decode from 0.1–1.0 m at $\pm 30^\circ$ yaw	Detection rate (%) vs. distance/angle sweep, camera screenshots
QR pose accuracy	Validate PnP translation/rotation against a ground-truth ruler measurement	Printed table of tvec error at known distances
Fallback detector (QReader)	Confirm background-thread fallback activates under occlusion and poor lighting	Terminal logs showing primary $\rightarrow$ fallback transitions
Pose filter stability	Verify EMA filter removes jitter without introducing unacceptable lag	Plot of raw vs. filtered tvec over time
ArUco gridboard detection	Confirm landmark detection in the operational arena at expected range	Topic echo of estimated pose, RViz overlay screenshot
YOLO logo classification	Validate correct company label per bay	Confusion matrix / per-class accuracy on a held-out image set

TABLE VII
NAVIGATION TESTS

Test	Objective	Evidence
Odometry accuracy	Measure open-loop straight-line and rotation error	Measured vs. commanded displacement table
Lidar obstacle detection	Confirm stop/replan when an obstacle is placed in the path	RViz costmap screenshot, logged /scan data
SLAM map quality	Evaluate map consistency after one full arena traverse	RViz map screenshot, loop-closure confirmation
Waypoint navigation	Drive to a sequence of known goals without human intervention	Foxglove/RViz trajectory overlay, position error at goal
Localization under ArUco correction	Compare SLAM pose vs. ArUco-corrected pose near landmarks	Logged pose error before/after landmark observation

and safety. Table VIII shows the test objectives and validation methods used for the forklift assembly.

5) *Voice Interface Tests*: Human-robot interaction via voice was validated for accuracy and latency. Table IX outlines the experiments conducted to ensure the robustness of the offline speech recognition system.

6) *Web Interface Tests*: The web dashboard and HMI were tested for real-time telemetry updates and manual override capabilities. Table X summarizes the validation criteria for the web interface.

7) *Full Integration Tests*: Integration tests were run in sequence, each building on the previous stage to ensure that the perception, navigation, and actuation layers worked harmoniously. Table XI describes the end-to-end mission validation scenarios.

8) *Summary*: Table XII provides a final consolidation of the testing results, highlighting the overall validation of the PuzzleBot architecture.

VIII. RESULTS AND DISCUSSION

The integration and testing of the PuzzleBot platform provided a comprehensive view of the system's performance

TABLE VIII
FORKLIFT MECHANISM TESTS

Test	Objective	Evidence
Full stroke	Command 0.0 m $\rightarrow$ 0.1575 m and back	Encoder position log, video
Positional accuracy	Verify fork reaches target height ± 2 mm	Encoder readback table at 5 target heights
Load test	Lift pallet prototype at rated weight	Encoder drift under load, motor current log
E-stop response	Cut motor power mid-stroke via software and hardware E-stop	Fork stops within one encoder tick, no PCB faults

TABLE IX
VOICE INTERFACE TESTS

Test	Objective	Evidence
Keyword recognition	Validate that mission-critical commands ("pick up", "deliver", "stop") are recognized in a noisy lab environment	Command log with recognized intent and confidence score
False-positive rate	Confirm unrelated speech does not trigger commands	Log of false triggers during a 5-minute ambient noise session
End-to-end latency	Measure time from utterance to ROS 2 topic publish	Timestamped audio-in and topic-publish logs
Language robustness	Test with multiple speakers and accents	Recognition accuracy table per speaker

TABLE X
WEB INTERFACE TESTS

Test	Objective	Evidence
Dashboard reachability	Access UI from a device on the same network	Browser screenshot showing live status
Manual override buttons	Confirm each button publishes the correct ROS 2 command	Topic echo while clicking, UI screenshot
Live telemetry display	Verify pose, forklift height, and camera feed update in real time	Screen recording of dashboard during robot motion
Concurrent users	Confirm two clients can view simultaneously without race conditions	Two-browser screenshot, no command duplication in logs

TABLE XI
FULL INTEGRATION TESTS

#	Test	Objective	Evidence
1	Pallet approach only	Robot detects QR code and stops at target_dist	Logged z_dist and goal_state flag
2	Pick-up sequence	Full open-loop insertion + forklift raise	Video, encoder log, timestamped state machine output
3	Navigation to delivery bay	Robot carries pallet from pick-up zone to YOLO-identified company bay	RViz/Foxglove trajectory, logo classification log
4	Deposit sequence	Forklift lowers, robot reverses, pallet left in bay	Video, encoder log
5	Full autonomous mission	Pick up $\rightarrow$ navigate $\rightarrow$ deliver, triggered by voice or web command, no human intervention	End-to-end demo video, full terminal/ROS 2 log

across different operational domains.

A. Main Achievements

The PuzzleBot project achieved the integration of a modular autonomous robotic architecture composed of software, hard-

ware, perception, navigation, embedded control, simulation, and intelligent interaction layers.

The main result was not a single isolated functionality, but the construction of a coordinated system in which multiple

TABLE XII
TEST SUMMARY

Test	Objective	Evidence
QR / ArUco detection	Validate visual perception pipeline	Camera screenshots, topic output, accuracy table
Pose filter validation	Confirm stability and lag of EMA filter	Raw vs. filtered pose plot
YOLO logo recognition	Validate delivery-site identification	Classification accuracy, confusion matrix
Forklift movement control	Validate embedded actuator control	Encoder / PWM log, load-test video
Odometry & SLAM	Validate autonomous mobility	RViz map and trajectory, position error table
Obstacle avoidance	Validate safety layer	Costmap screenshot, stop-event log
Voice command recognition pipeline	Validate basic HRI command	Command logs, recognition accuracy table
Web dashboard	Validate HMI communication	UI screenshot, topic echo log
End-to-end mission	Validate full system integration	Demo video, complete ROS 2 bag file

subsystems contributed to the execution of warehouse oriented robotic tasks.

One of the main achievements was the development of a modular ROS 2 based architecture. The system was organized into specialized functional layers for perception, navigation, control, robot description, hardware interfaces, mission behavior, and communication. This modular structure allowed different components to exchange information through shared interfaces while preserving separation of responsibilities. As a result, the robot could process sensor data, estimate its state, receive mission commands, generate navigation goals, and execute motion through a coordinated robotic pipeline.

The perception layer was also successfully integrated into the general architecture. The robot incorporated camera based perception for QR detection, ArUco marker detection, visual pose estimation, and object recognition. These perception capabilities were important because they allowed the robot to extract task relevant information from the environment instead of relying only on predefined map positions. QR and fiducial markers supported identification, localization correction, and pallet alignment, while object detection contributed to recognizing relevant elements in the warehouse scenario.

Another relevant achievement was the implementation of autonomous navigation and control. The robot used map based planning, waypoint selection, trajectory following, and local maneuver execution to move through the working environment. A planning layer determined where the robot should move, while the control layer translated those planned routes into executable velocity commands. Pure Pursuit supported

trajectory tracking, while open loop control was used for short and precise movements required during pallet handling, such as entering or leaving a pallet area.

The finite state machine represented a major achievement at the mission level. Instead of limiting the robot to independent navigation or perception actions, the FSM coordinated a complete task sequence that included waiting for a mission, calculating a route, selecting the next waypoint, moving toward target zones, searching for QR codes, aligning with a pallet, activating the forklift, moving toward the trailer zone, identifying the target destination, and completing the delivery

behavior. This demonstrated autonomy at the task level rather than only at the motion level. Fig. 10 shows the active mission tracking and state transitions during a successful delivery run.

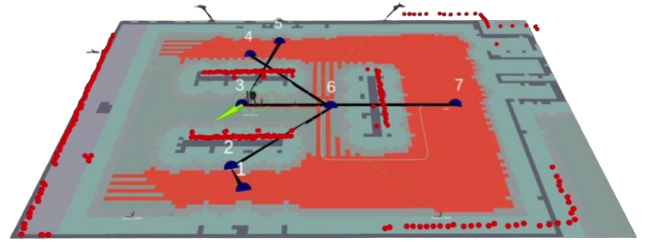


Fig. 10. Visualization of the autonomous mission FSM during a waypoint-based navigation and pallet handling task.

The embedded forklift control subsystem was another key contribution. The system integrated a geared DC motor, encoder feedback, PWM based actuation, FPGA control, and SPI communication with the main computing layer. The forklift mechanism connected mechanical motion with the autonomous software stack, allowing high level mission decisions to be converted into physical lifting actions. This integration showed that the robot was capable of coordinating mobile navigation with manipulation.

Communication with the FPGA was also validated as part of the embedded control architecture. The high level computing layer generated commands or setpoints, while the FPGA handled low level motor control, PWM signal generation, motor direction logic, and encoder processing. This separation improved the system architecture by assigning computationally intensive tasks to the main processor and deterministic actuator control to the embedded layer.

The project also achieved intelligent interaction through an offline voice interface. The voice system used local audio processing, voice activity detection, MFCC feature extraction, Hidden Markov Model based recognition, a wake word finite state machine, and a ROS 2 bridge to publish motion commands. This enabled the robot to receive basic movement instructions without depending on cloud based speech recog-

dition services, improving privacy, latency, and robustness in environments with limited connectivity.

A web based human machine interface was also incorporated for robot supervision and control. The HMI displayed relevant robot information such as warehouse map, robot pose, velocity, voice commands, and lifter position. This made autonomous behavior more observable, auditable, and interruptible, allowing the operator to verify the robot state and intervene when necessary.

Visualization and debugging tools represented another important achievement. Foxglove, RViz, and perception dashboards were used to inspect robot state, sensor data, odometry, maps, transforms, command messages, and camera based detections. These tools improved the team's ability to diagnose integration problems and validate whether perception, localization, navigation, control, and actuation were communicating correctly.

Finally, the project incorporated simulation through a Gazebo digital twin of the robot and the warehouse environment. The digital twin included the PuzzleBot platform, forklift mechanism, sensors, racks, conveyors, pallets, QR based objects, ArUco markers, LiDAR, cameras, physical dynamics, inertias, limits, and ROS 2 control interfaces. This simulation environment allowed the team to validate navigation,

perception, control, and mission behavior before physical deployment, reducing risk during real robot testing.

Overall, the main achievement of the project was the integration of a complete robotic system. PuzzleBot combined autonomous navigation, visual perception, mission level coordination, embedded forklift control, FPGA communication, voice interaction, web supervision, visualization tools, teleoperation, and simulation into a single architecture. This demonstrated that the system was developed as an integrated autonomous robot rather than as a collection of unrelated subsystems.

B. Technical Limitations

Physical Constraints

The PuzzleBot platform presented mechanical challenges regarding structural robustness and payload capacity. To maintain operational stability and prevent power-related resets due to overcurrent

("brownouts"), we were required to constrain the robot's maximum velocities. Furthermore, the compact physical footprint of the chassis limited the design flexibility for the forklift mechanism. Specifically, integrating the third actuator necessitated mounting configurations that, while functional for this project, may present long-term structural risks.

Computational Constraints

The onboard Jetson Nano possesses limited computational resources, which necessitated offloading certain processing tasks to an auxiliary computer via a wireless connection. This architectural dependency constrained the robot's true autonomous capabilities, as it relied on external processing power to successfully execute complex navigation and perception tasks.

Network and Bandwidth Constraints

The limited bandwidth of the wireless communication link between the onboard Jetson Nano and the offboard computer introduced constraints on data transmission. We were required to implement a strategic data management plan, prioritizing critical telemetry and control signals to minimize packet loss and reduce latency during mission execution.

Testing and Validation Constraints

Throughout the development cycle, limited access to the physical robotic platform hampered end-to-end system validation. While high-fidelity simulations in Gazebo mitigated this challenge for the navigation stack, we encountered late-stage interfacing issues that prevented a comprehensive physical demonstration of the fully integrated system. Additionally, logistical constraints, specifically the shared access to the single test environment among eleven different teams, limited the duration and frequency of physical integration testing.

C. Project Management and Collaboration Methodology

The development of the PuzzleBot was managed following a structured technological design approach, specifically employing an Agile-based framework organized into iterative Sprints. This methodology supported the STE0104C subcompetency by allowing for continuous integration and rapid prototyping.

GitHub served as the central project management hub, where the distributed architecture was managed across multiple repositories. We utilized Git for version control, branch-based development for feature isolation, and issue tracking for task management. This organizational structure ensured that the software, perception, and embedded layers could progress in parallel while maintaining a unified integration path. Weekly synchronization meetings were used to align technical interfaces between the high-level ROS 2 stack and the low-level FPGA control, ensuring that cross-functional dependencies were resolved early in the design cycle.

IX. AUTHOR CONTRIBUTIONS

In accordance with the project requirements, the specific technical contributions of each team member are summarized below:

- **Juan Manuel Félix Campos:** Lead developer of the Finite State Machine (FSM), global navigation algorithms, and the high-fidelity Gazebo simulation environment.
- **José Luis Urquieta Aguilar:** Designer of several robotic mechanisms and lead developer of the Human-Machine Interface (HMI), including the web-based dashboard and the offline voice interface.
- **Alejandro Rodríguez del Bosque:** Designer of robotic mechanisms, contributor to the FPGA embedded layer, and developer of navigation components, specifically implementing the Bug 2 obstacle avoidance algorithm.
- **Samuel Francisco Cabrera Ramírez:** Designer of several robotic mechanisms and lead developer of local navigation algorithms for precision pallet alignment and fiducial-based visual servoing.

- **Eduardo Hernandez Valdez:** Lead developer of the electronic hardware integration, FPGA-based control subsystem (PicoRV32), and the SLAM and Monte Carlo Localization (MCL) pipelines.

A. Lessons Learned

One of the most important lessons learned during the PuzzleBot project was that autonomous robotics integration must be treated as a system level problem from the beginning. Individual modules can work correctly in isolation, but the complete robot only becomes reliable when perception, navigation, control, embedded hardware, communication, simulation, and user interfaces are validated together. The project showed that the main difficulty was not only developing algorithms, but ensuring that each subsystem exchanged information correctly, responded within acceptable timing constraints, and behaved consistently on both the digital twin and the physical robot.

The use of Gazebo as a simulation environment was valuable because it allowed the team to test the robot, forklift mechanism, sensors, and warehouse environment before physical deployment. However, the project also demonstrated that simulation cannot fully replace real world validation. Mechanical friction, sensor noise, communication instability, actuator limitations, power issues, and physical alignment errors can only be fully understood through experiments on the real platform.

A critical lesson was the importance of power distribution in robotic systems. During integration, the team identified instability related to the electrical supply of the computing and actuation layers. The issue was resolved by powering the Hackerboard and the Jetson from separate power sources, while supplying the lifter mechanism independently at 5 V. This separation reduced interference between high level computing components and actuator loads, improving the stability of the system during operation. The experience showed that power architecture is not only an electrical detail, but a fundamental requirement for reliable software and hardware integration.

Talking about voltages as well as the integration of the electronics, we thought that it would be easier. We were really inclined into using simpler systems, sensors and actuators, but I think in the end it would have been way easier to use a 360 servo instead of a DC motor with a relative encoder. Our mentality at first was that we could fix everything in software, which did work at first, but eventually got way worse with all of the edge cases that surged. In the end, most of them were solved, but the development of the electronics and mechanism could have been way easier if we had spent more time and resources on what we previously mentioned.

Another key lesson was the importance of observability and recovery mechanisms. Tools such as the web HMI, Foxglove, RViz, voice interface, teleoperation tools, and priority command mux were not secondary features; they were necessary to inspect, interrupt, debug, and recover the robot during integration. In autonomous robotics, a system should not only be able to execute a mission, but also allow the operator

to understand what is happening and safely intervene when needed.

Overall, the project reinforced that building an autonomous robot requires disciplined modular design, early integration testing, clear communication interfaces, stable power distribution, and continuous validation between software and hardware. Future development should prioritize more systematic end to end testing, stronger documentation, earlier physical integration of all subsystems, and a more robust power management strategy for computing, sensing, and actuation components.

Juan Manuel Félix Campos

Through this project, for the first time in my career as a robotics engineer, I successfully applied the necessary knowledge to build a truly autonomous robotics platform. Specifically, we developed a differential mobile robot adapted to serve as a physical twin of an automated industrial forklift for the enterprise E80.

By implementing mapping and localization using SLAM and Monte Carlo methods, I learned how a probabilistic approach handles sensor uncertainty. This allows the system to converge on an accurate pose estimate within both known and unknown environments, utilizing integrated data from LiDAR, odometry, and camera sensors. Once the robot's pose in the map was established, I understood how to plan routes and execute them using controllers, alongside obstacle avoidance strategies like costmaps and the Bug2 algorithm. Furthermore, after navigating a route, we utilized "local" controllers and perception modules relying on camera vision for precision tasks such as alignment and object identification.

I also learned how to orchestrate these complex, fully autonomous operations using tools like Finite State Machines (FSMs) and Behavior Trees, closely adhering to the ROS 2 Nav2 standard. During development, my primary focus was simulating the robot and its environment using Gazebo and ROS 2. This experience provided profound insights into modeling a complete robotic system alongside its dynamic environment. It also taught me how to structure a robotics project architecture following best development practices to avoid technical debt.

Jose Luis Urquieta Aguilar

This project was one of the most complete integration experiences I have had in robotics because it showed me that building an autonomous robot is not only about programming isolated algorithms or designing individual mechanisms. Throughout the PuzzleBot project, I learned that a real robotic system depends on the correct interaction between perception, localization, navigation, embedded control, mechanical design, simulation, communication, and human supervision. A subsystem can work correctly by itself, but the real challenge appears when all components must operate together on the physical robot.

One of the most important lessons I learned was that integration must be planned from the beginning. It is not enough to develop separate modules and connect them at the end. Each part of the system needs clear interfaces, reli-

able communication, stable power distribution, and constant validation. The project also taught me that simulation and physical testing complement each other. Gazebo allowed us to test the robot and the warehouse environment in a controlled digital twin, but the physical robot revealed problems that simulation cannot fully reproduce, such as mechanical friction, power instability, sensor alignment, actuator limitations, and communication issues.

Personally, I contributed mainly to the human machine interface, voice detection system, and forklift integration. My work focused on improving how the user could interact with the robot, monitor its behavior, and send commands through intelligent interfaces. The voice interface allowed the robot to receive offline movement commands, while the HMI helped make the system more observable and easier to supervise. I also contributed to the forklift integration, which helped connect the interaction and control layers with the physical manipulation task.

Each team member contributed to a different technical layer of the system. Eduardo focused on the forklift subsystem, SLAM, and Monte Carlo Localization. His work contributed both to the physical manipulation capability of the robot and to the localization strategy required for autonomous operation. Samuel worked on pallet alignment and forklift development, helping the robot connect visual perception with mechanical manipulation. His contribution was important for aligning the robot with the pallet and supporting the pickup process. Juan worked on simulation, the finite state machine, YOLO based perception, and module integration. His contribution was essential because the FSM connected the mission logic with navigation, perception, and manipulation, while the simulation environment allowed the system to be tested before physical deployment. Alejandro focused on the forklift mechanism and obstacle avoidance, contributing to the robot's ability to interact with the payload and react to obstacles during navigation. His work supported both the mechanical and safety related parts of the project.

From my perspective, the strongest result of the team was that each contribution supported a different part of the same architecture. The project was not only a collection of independent tasks; it became a system where localization, perception, navigation, manipulation, simulation, embedded control, and human interaction had to work together. This helped me understand that successful robotics engineering requires both technical specialization and system level thinking.

Overall, this project helped me grow as an engineer because it forced me to think beyond my own module. I learned that a robot must be observable, controllable, interruptible, and recoverable, especially when working with real hardware. The experience also reinforced the importance of teamwork, documentation, testing, and communication between subsystems. Although the project had integration challenges, it gave us a much deeper understanding of what it means to build an autonomous robotic platform from multiple hardware and software layers.

Alejandro Rodríguez del Bosque

The PuzzleBot challenge served as the definitive culmination of my undergraduate engineering journey. While the technical scope of the project was vast, spanning computer vision, ROS 2 middleware, FPGA integration, and autonomous navigation, the true challenge lay in the collaborative integration of these disparate domains.

Working as a team, I quickly realized the importance behind writing code and organizing smaller subsystems into a larger, robust environment. We were tasked with managing a distributed robotic architecture where specialized modules for perception, localization, and embedded control had to communicate seamlessly. This experience highlighted the vital importance of concurrent systems. In a robot, the luxury of sequential execution does not exist. We had to develop software that could simultaneously process camera streams, update a global costmap, calculate navigation trajectories, and manage low-level motor commands for the forklift mechanism. Coordinating these concurrent processes within a shared ROS 2 ecosystem taught me how to manage data flow, handle latency, and ensure that our subsystems remained synchronized, even under the pressure of real-time execution.

The transition from theoretical design to real-world deployment was the project's most transformative phase. Simulation in Gazebo provided a valuable foundation, but it could not replicate the unpredictable nature of physical reality. Deploying the PuzzleBot in a real context forced us to confront variables that models often omit: lighting fluctuations affecting QR code detection, mechanical friction within the forklift assembly, power instability, and real-world sensor noise. Learning to debug and stabilize a system on physical hardware was a grueling but essential process. It bridged the gap between academic theory and professional practice, showing me that the "best" algorithm on paper is useless if the hardware cannot support it.

This capstone project truly integrated my four years of college into a final, tangible result. Every course I completed, from digital logic and control theory to software architecture and algorithm analysis, suddenly found its purpose. It was the moment where my education crystallized into a single, functional machine. It required me to stop thinking as a student of isolated subjects and start thinking as a systems engineer, capable of understanding the delicate interplay between firmware, software, and mechanical constraints.

Reflecting on this experience, I feel a deep sense of accomplishment. The PuzzleBot challenge did more than test my technical knowledge; it reshaped my professional identity. It validated that I am ready to transition from the classroom to the field, equipped with the resilience to troubleshoot complex hardware-software integration issues and the systemic vision to build autonomous platforms that work in the real world. This project was not merely a graduation requirement; it was the proving ground that synthesized my past four years into the engineer I am today.

Eduardo Hernandez Valdez

This project represented a major milestone in my academic and professional journey, serving as a rigorous proving ground

where I was able to bridge the profound gap between high-level probabilistic theory and deterministic hardware execution. My primary technical focus was centered on the localization and navigation architecture, a challenge that required the implementation of a robust Simultaneous Localization and Mapping (SLAM) system and a high-performance, GPU-accelerated Monte Carlo Localization (MCL) filter.

Developing the SLAM pipeline, specifically utilizing a CoreSLAM-based approach, taught me the critical importance of handling real-world sensor uncertainty. By implementing likelihood fields and bilinear interpolation for scan-matching, I learned how to transform noisy Lidar data into sharp, consistent occupancy grids that could be used for mission-critical navigation. This process was not merely about writing code; it was an exercise in empirical tuning, as I used Genetic Algorithms to optimize hyperparameters and minimize map entropy.

The implementation of the Monte Carlo Localization system was perhaps the most technically demanding aspect of my contribution. Transitioning from a CPU-bound particle filter to a GPU-accelerated architecture using custom CUDA kernels via CuPy was a transformative experience. It required a deep dive into parallel computing to ensure that thousands of particles could be evaluated in real-time, allowing the PuzzleBot to maintain centimetric pose accuracy even when navigating through dynamic and feature-poor warehouse environments.

Beyond the high-level software stack, I also dedicated significant effort to the embedded layer, specifically focusing on the FPGA-based control subsystem. Working with programmable logic and the PicoRV32 softcore was eye-opening; it required a shift in mindset to account for hardware-level timing, clock cycles, and the strict determinism needed to manage motor PWM drivers and quadrature encoder feedback over a high-speed SPI link. This dual exposure—managing both the "cloud" of probabilistic particles and the "iron" of hardware logic—taught me that a truly autonomous robot is only as strong as its weakest interface. Seeing these disparate layers converge into a single, functional platform during the final autonomous mission was the most rewarding and intellectually fulfilling aspect of my engineering journey.

Samuel Francisco Cabrera Ramírez

This project helped me understand that autonomous robotics depends on the correct integration of perception, control, and mechanical actuation. My main contribution focused on pallet alignment and forklift development, where I learned how visual information can be transformed into precise robot motion.

Working on pallet alignment showed me that detecting a QR code or marker is only the first step. The real challenge is using that information to estimate the pallet position, align the robot correctly, and generate useful movement commands. This helped me understand the importance of camera calibration, pose estimation, coordinate frames, and control logic in precision robotics.

The forklift subsystem also taught me that mechanical design requires constant testing and iteration. A lifting mech-

anism must consider torque, friction, stability, actuator response, and real physical constraints. Overall, this project helped me grow as a robotics engineer because it showed me that a robot must not only perceive and navigate, but also interact reliably with its environment.

X. MAPPING BETWEEN COURSE MODULES AND PROJECT CONTRIBUTIONS

The alignment between the academic requirements and the technical achievements of the project is summarized in Table XIII.

XI. CONCLUSION

The PuzzleBot project demonstrated the development and integration of a modular autonomous robotic architecture that combined perception, navigation, control, embedded systems, simulation, and intelligent interaction interfaces. Rather than focusing on isolated algorithms or independent subsystems, the project aimed to connect multiple technical layers into a coordinated mobile robot capable of operating in a warehouse automation scenario.

The implemented architecture integrated ROS 2 based communication, visual perception, localization, path planning, finite state machine behavior, forklift actuation, FPGA based motor control, web supervision, offline voice commands, visualization tools, teleoperation, and simulation. This allowed the robot to be

designed as a complete robotic platform where high level autonomy, low level actuator control, and human supervision could interact within the same system. The Gazebo digital twin also provided an important validation environment by allowing the team to test the robot, forklift mechanism, sensors, and warehouse environment before deploying the same logic on the physical platform.

One of the main technical contributions was the connection between the autonomous software stack and the embedded control subsystem. The forklift mechanism required coordination between mechanical design, motor actuation, encoder feedback, PWM generation, FPGA logic, and communication with the main computing layer. In parallel, the interaction layer strengthened the system by making autonomous behavior observable, controllable, interruptible, and recoverable through the web HMI, voice interface, priority command mux, Foxglove, RViz, and teleoperation tools.

The main challenge of the project was not the implementation of a single component, but the physical integration and end to end validation of the complete system. Robotic integration required aligning software behavior with real sensors, mechanical constraints, communication reliability, actuator response, localization accuracy, and user intervention mechanisms. Although some modules still require additional experimental validation and robustness testing, the work completed establishes a strong foundation for a scalable autonomous robotic system.

Overall, the PuzzleBot project showed that autonomous robot development depends not only on individual algorithms,

TABLE XIII
MAPPING BETWEEN COURSE MODULES AND PROJECT CONTRIBUTIONS

Course Module	Evidence in PuzzleBot	Why It Matters
Module 1: Emerging Technologies	ROS 2, computer vision, YOLO, ArUco, QR, Docker, Foxglove	Demonstrates use of modern robotics and AI-related tools
Module 2: Embedded Systems	FPGA, PWM, encoder, SPI, IMU, PCB design	Demonstrates low-level hardware integration
Module 3: Autonomous Robots	Odometry, localization, navigation, path planning, FSM, pallet alignment	Demonstrates autonomous behavior and robot navigation
Module 4: Intelligent Interfaces	Voice interface, web dashboard, rosbridge, teleoperation, visualization	Demonstrates human-robot interaction and monitoring

but on the correct integration of perception, control, navigation, embedded hardware, simulation, and intelligent interfaces. The resulting architecture provides a solid base for future improvements in autonomy, manipulation, safety, and warehouse oriented robotic operation.

REFERENCES

- [1] Slamurais, "puzzlebot_software," GitHub repository.
- [2] Slamurais, "puzzlebot_embedded," GitHub repository.
- [3] Slamurais, "voice_user_interface," GitHub repository.
- [4] Slamurais, "Web-interface-for-PuzzleBot," GitHub repository.
- [5] Slamurais, "puzzlebot_localization," GitHub repository.
- [6] Slamurais, "puzzlebot_comms," GitHub repository.
- [7] Slamurais, "ROSNin_Electronics," GitHub repository.
- [8] IEEE Author Center, "Authoring Tools and Templates."
- [9] P. Corke, *Robotics, Vision and Control: Fundamental Algorithms in Python*, 3rd ed. Cham, Switzerland: Springer Nature Switzerland, 2023.
- [10] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, and M. J. Marín-Jiménez, "Automatic generation and detection of highly reliable fiducial markers under occlusion," *Pattern Recognition*, vol. 47, no. 6, pp. 2280–2292, 2014.
- [11] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.
- [12] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA: MIT Press, 2005.
- [13] B. Steux and O. El Hamzaoui, "coreSLAM: a SLAM algorithm in less than 200 lines of C code," in *Proc. 11th Int. Conf. Control, Automation, Robotics and Vision (ICARCV)*, 2010, pp. 1975–1979.
- [14] R. Okuta, Y. Unno, D. Nishino, S. Hido, and C. Loomis, "CuPy: A NumPy-compatible library for NVIDIA GPU calculations," in *Proc. Workshop on Machine Learning Systems (LearningSys) at NeurIPS*, 2017.
- [15] P. Virtanen *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.
- [16] N. Koenig and A. Howard, "Design and use paradigms for Gazebo, an open-source multi-robot simulator," in *Proc. IEEE/RSJ IROS*, 2004, pp. 2149–2154.
- [17] E. Rohmer, S. P. N. Singh, and M. Freese, "V-REP: A versatile and scalable robot simulation framework," in *Proc. IEEE/RSJ IROS*, 2013, pp. 1321–1326.
- [18] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, real-time object detection," in *Proc. IEEE CVPR*, 2016, pp. 779–788.
- [19] S. Macenski and I. Jambrecic, "SLAM Toolbox: SLAM for the dynamic world," *Journal of Open Source Software*, vol. 6, no. 61, p. 2783, 2021.
- [20] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.
- [21] S. Macenski *et al.*, "The Marathon 2: A navigation system," in *Proc. IEEE/RSJ IROS*, 2020.
- [22] NVIDIA Corporation, "Jetson Nano Developer Kit," Technical Brief, 2019.
- [23] Lumelsky, V. J., & Stepanov, A. A. (1987). Path-planning strategies for a point mobile automaton moving amidst unknown obstacles of arbitrary shape. *Algorithmica*, 2(1-4), 403-430.
- [24] Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100-107.
- [25] P. J. Besl and N. D. McKay. (1992). A method for registration of 3-D shapes. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 14(2), 239-256.
- [26] J. H. Holland. (1992). *Adaptation in natural and artificial systems: An introductory analysis with applications to biology, control, and artificial intelligence*. MIT Press.
- [27] J. Bradbury, R. Frostig, P. Hawkins, J. P. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang. (2018). JAX: composable transformations of Python+NumPy programs. [Online]. Available: <http://github.com/google/jax>.
- [28] C. Stachniss, G. Grisetti, and W. Burgard. (2005). Information Gain-based Exploration of Unknown Environments with Multi-Robot Systems. In *Proc. of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 249-254.
- [29] C. Wolf. (2015). PicoRV32 - A Size-Optimized RISC-V CPU. [Online]. Available: <https://github.com/YosysHQ/picorv32>.
- [30] D. Dolgov, S. Thrun, M. Montemerlo, and J. Diebel. (2010). Path planning for autonomous vehicles in unknown semi-structured environments. *The International Journal of Robotics Research*, 29(4), 485-501.
- [31] D. V. Lu, D. Hershberger, and W. D. Smart. (2014). Layered costmaps for context-aware navigation. In *Proc. IEEE/RSJ IROS*, pp. 709-715.
- [32] R. Coulter. (1992). *Implementation of the Pure Pursuit Path Tracking Algorithm*. Carnegie Mellon University, Robotics Institute.